

SQIsign v2 Specification Review

Feedback from an Independent Rust Implementation

Deirdre Connolly

Selkie Cryptography

durumcrustulum@gmail.com

April 15, 2026 (living document)

Abstract

Each item below describes something that the SQIsign specification [1] (version 2025-07-07) leaves unstated, ambiguous, or implicit, and that we could only resolve by inspecting the C reference implementation. We file these as constructive suggestions so that future implementors can achieve interoperability from the spec alone.

Contents

1	Severity scale	2
2	$\mathbb{F}_{p^2}$ square root canonicalization	2
3	ProductToTheta coordinate convention	2
4	“Squared theta” definition	3
5	hadamard_bool in the (2, 2)-chain	3
6	Torsion basis slot convention	3
7	Normalization of $(A_{24} : C_{24})$	3
8	Gluing requires Jacobian y-coordinates	3
9	Jacobian doubling for balanced strategy	4
10	DIFFERENCEPOINT normalization factor	4
A	Never recompute $P-Q$ via DIFFERENCEPOINT	5
B	Change-of-basis matrix application convention	5
C	GENERALIZEDREPRESENTINTEGER sampling ranges and divisibility	6
D	Response lattice: product vs. intersection	7
E	FIXEDDEGREEISOGENY missing u^{-1} normalization	7
F	Fixed-precision Hermite Normal Form	7

1 Severity scale

CRITICAL	The spec as written produces a non-interoperable implementation: verification rejects valid signatures, or signing produces unverifiable ones. The spec must be amended.
HIGH	An implementor will almost certainly write a bug that passes unit tests but fails KAT or cross-implementation testing. Debugging requires reading the C reference source.
MEDIUM	An implementor may write a bug depending on reasonable but wrong assumptions about conventions. A short note eliminates the ambiguity.
LOW	The spec is correct but incomplete. A remark or forward pointer would help.

2 $\mathbb{F}_{p^2}$ square root canonicalization

Affects Algorithm 8.12 (*DIFFERENCEPOINT*), and transitively every algorithm that consumes a torsion basis. **CRITICAL**

Algorithm 8.12 says “compute $\delta = \sqrt{B_{XZ}^2 - B_{XX}B_{ZZ}}$ ” without specifying which of the two square roots in $\mathbb{F}_{p^2}$ to use. The two roots δ and $-\delta$ produce two *affinely distinct* points x_{P-Q} . These are not projectively equivalent; they are genuinely different points. The wrong choice cascades through LADDER3PT (wrong kernel generator), the challenge isogeny (different curve E_{chl}), and every subsequent verification step. In our implementation the j -invariant of E_{chl} was completely wrong until we matched the C reference’s canonicalization.

The spec should define a canonical $\mathbb{F}_{p^2}$ square root. The C reference uses the convention from Aardal et al. [2]: negate the output so that the real part (as an integer in $[0, p)$) is even; if the real part is zero, negate so the imaginary part is even. This should be stated in §8.1 (field arithmetic) as a normative requirement. A note of the form

Throughout this document, “compute $\sqrt{a}$ ” for $a \in \mathbb{F}_{p^2}$ means the unique root r such that $r^2 = a$ and $\text{Re}(r) \bmod 2 = 0$ (or, if $\text{Re}(r) = 0$, $\text{Im}(r) \bmod 2 = 0$).

would suffice.

3 ProductToTheta coordinate convention

Affects Algorithm 8.37 (*PRODUCTToTHETA*). **HIGH**

The product theta coordinates are described abstractly via level-2 theta functions. The mapping from Montgomery projective coordinates $(X : Z)$ to product theta coordinates is not stated explicitly. The natural “theta-like” representation for Montgomery arithmetic is $(X - Z, X + Z)$, which appears throughout the isogeny literature. The C reference uses raw (X, Z) directly. Using $(X - Z, X + Z)$ produces a completely wrong change-of-basis matrix N .

Algorithm 8.37 should state explicitly that the product theta coordinates are formed as

$$(a, b, c, d) = (X_1X_2, X_1Z_2, Z_1X_2, Z_1Z_2)$$

from Montgomery projective pairs $(X_1 : Z_1), (X_2 : Z_2)$.

4 “Squared theta” definition

Affects Algorithm 8.29, Algorithm 8.34, Algorithm 8.38.

High

Several algorithms refer to “squared theta coordinates” without a centralized definition. In the C reference, `to_squared_theta(P)` computes component-wise squaring *followed by a Hadamard transform*: $\tilde{P} = \mathcal{H}(P^2)$. The name suggests only squaring. We omitted the inner Hadamard in the generic isogeny evaluation, computing $\mathcal{H}(\text{inv} \cdot P^2)$ instead of the correct $\mathcal{H}(\text{inv} \cdot \mathcal{H}(P^2))$.

Add a definition early in §8.5:

*The **squared theta** of a point $P = (a : b : c : d)$ is $\tilde{P} = \mathcal{H}(a^2, b^2, c^2, d^2)$, i.e. component-wise squaring followed by the Hadamard transform.*

5 hadamard_bool in the (2, 2)-chain

Affects Algorithm 8.47 (ISOGY22CHAINWITHTORSION).

High

The algorithm describes each generic step uniformly. The C reference uses three formula variants controlled by boolean flags:

- Normal ($i < n-2$): codomain Hadamard-transformed; $\text{eval} = \mathcal{H}(\text{precomp} \cdot \mathcal{H}(P^2))$.
- Penultimate ($i = n-2$): codomain in dual form; $\text{eval} = \text{precomp} \cdot \mathcal{H}(P^2)$.
- Ultimate ($i = n-1$): extra input Hadamard, codomain dual; $\text{eval} = \text{precomp} \cdot \mathcal{H}(\mathcal{H}(P)^2)$.

The splitting step expects dual form. Using the normal formula for all steps feeds the splitter a Hadamard-transformed null point with no zero $U_{i,j}(0)$ entry, causing verification to fail after an otherwise correct 125-step chain.

A short remark in Algorithm 8.47 noting that the last two steps omit the final Hadamard (and the ultimate step pre-Hadamards the input) would prevent this.

6 Torsion basis slot convention

Affects Algorithm 2.2 (TORSIONBASISFROMHINT).

MEDIUM

The spec says the algorithm returns $(P, Q, P-Q)$. The C reference stores $P-Q$ in the `Q` slot and Q in `PmQ`, so `LADDER3PT` computes $P + [m](P-Q)$, not $P + [m]Q$. The swap ensures the multiplied point avoids $(0, 0)$ but is not documented.

State the output convention explicitly, or note that `LADDER3PT` is called with the difference in the `Q` position.

7 Normalization of $(A_{24} : C_{24})$

Affects Algorithm 2.2 (TORSIONBASISFROMHINT).

MEDIUM

The C reference normalizes to $((A+2)/4 : 1)$ before basis generation. If the curve arrives from an isogeny codomain with $C_{24} \neq 1$, the Montgomery ladder produces a different projective representative, which propagates through `DIFFERENCEPOINT` and `LADDER3PT`.

Note in Algorithm 2.2 that the curve must be in normalized form before computing the basis.

8 Gluing requires Jacobian y -coordinates

Affects Algorithm 8.39 (GLUINGEVAL).

Low

Montgomery x -only arithmetic cannot distinguish $P + Q$ from $P - Q$. The gluing evaluation needs this distinction. The C reference lifts to Jacobian (x, y, z) via Okeya–Sakurai for this step—the only place in verification that needs y -coordinates. Note this in Algorithm 8.39.

9 Jacobian doubling for balanced strategy

Affects Algorithm 8.47, Phase 1.

Low

The spec does not specify whether Phase 1 doublings use Montgomery or Jacobian coordinates. The C reference doubles in Jacobian, then converts via `jac_to_xz`: $(x, y, z) \mapsto (x, z^2)$. The theta coordinate computations are sensitive to the projective representative. Note this, or at minimum state that the representative matters.

10 DIFFERENCEPOINT normalization factor

Affects Algorithm 8.12 (DIFFERENCEPOINT).

Low

The C reference multiplies B_{XX} , B_{XZ} , B_{ZZ} by $C \cdot \overline{C}^2 \cdot \overline{Z_P}^2 \cdot \overline{Z_Q}^2$ (Galois conjugation) before the quadratic solve. This makes the discriminant a fourth power in $\mathbb{F}_p$, reducing the $\mathbb{F}_{p^2}$ sqrt to an $\mathbb{F}_p$ sqrt. We initially used $(Z_P Z_Q)^2$, which lacks this property since $\overline{z}^2 \neq z^2$ in general. State the normalization factor and its purpose in Algorithm 8.12.

References

- [1] SQIsign team, *SQIsign: Compact Post-Quantum Signatures from Quaternions and Isogenies*, Specification version 2025-07-07, <https://sqisign.org/spec/sqisign-20250707.pdf>.
- [2] M. N. H. Aardal, M. P. Azimova, E. L. Carrión, L. Dillinger, and M. J. Kannwischer, *Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4*, Cryptology ePrint Archive, Paper 2024/1563, <https://eprint.iacr.org/2024/1563>.
- [3] C. Costello, H. Hisil, and B. Smith, *Faster Compact Diffie–Hellman: Endomorphisms on the x-line*, ASIACRYPT 2017, <https://eprint.iacr.org/2017/518>.

A Never recompute $P-Q$ via DIFFERENCEPOINT

Affects Algorithm 4.9 (verification), lines 11–26, and transitively the $(2, 2)$ -isogeny chain. **CRITICAL**

A torsion basis $(P, Q, P-Q)$ is produced by TORSIONBASISFROMHINT (Algorithm 2.2). The third component $P-Q$ is computed once, via DIFFERENCEPOINT, which internally takes an $\mathbb{F}_{p^2}$ square root. Subsequent operations—scaling (repeated doubling), matrix application (Algorithm 4.9 line 14), and the even response isogeny (line 17)—must propagate all three points together. If at any point $P-Q$ is recomputed from P and Q via DIFFERENCEPOINT, the fresh square root may pick the opposite branch, producing a point that is *affinely distinct* from the original $P-Q$.

Concretely, the two branches of DIFFERENCEPOINT yield $x(P-Q)$ and $x(P+Q) = x(2P-Q)$; these are different points. All subsequent differential additions (the three-point ladder, the biladder for matrix application, and the $(2, 2)$ -isogeny chain’s LADDER3PT and LIFT_BASIS) consume $P-Q$ as a third input and produce wrong results if it is silently replaced by $2P-Q$.

In our implementation, three separate recomputations caused failures:

1. After scaling the basis by repeated doubling, we reconstructed $P-Q$ from the doubled P and Q instead of doubling $P-Q$ alongside them.
2. The matrix application $M_{\text{chl}} \cdot (P, Q)$ computed $P' - Q'$ via DIFFERENCEPOINT(P', Q') instead of a third biladder call $[(a-b)]P + [(c-d)]Q$.
3. Before entering the $(2, 2)$ -chain, we recomputed $P-Q$ from the post-isogeny images instead of pushing the original $P-Q$ through the isogeny as a third evaluation point.

Each of these independently caused KAT verification failure. Fixing all three was necessary and sufficient to make KAT vector 0 pass.

Current spec text. The spec does not warn against recomputing $P-Q$. Algorithms that scale or transform a basis mention only P and Q ; the third point $P-Q$ is not discussed.

Recommendation. Add a normative note in §2.2 or §8.2:

Once a basis $(P, Q, P-Q)$ has been produced by TORSIONBASISFROMHINT, the third point $P-Q$ must be propagated through every subsequent operation (doubling, matrix application, isogeny evaluation) alongside P and Q . Recomputing $P-Q$ via DIFFERENCEPOINT is not permitted: the square root may select a different branch, yielding an affinely distinct point that silently corrupts all downstream differential additions.

B Change-of-basis matrix application convention

Affects Algorithm 4.8 (SETCHANGEOFBASISMATRIX), Algorithm 4.9 (verification, line 14). **HIGH**

The spec does not specify whether the change-of-basis matrix $\mathbf{M}_{\text{chl}}$ is applied to a torsion basis (P, Q) by rows or by columns.

What the spec says. Algorithm 4.8 line 6 writes $(P_2, Q_2) \leftarrow \mathbf{M}_1 \cdot (P_2, Q_2)$ without specifying the convention. Algorithm 4.9 line 14 writes $(P_{\text{chl}}, Q_{\text{chl}}) \leftarrow \mathbf{M}_{\text{chl}} \cdot (P_{\text{chl}}, Q_{\text{chl}})$, again without specifying.

What the C reference does. `matrix_scalar_application_even_basis` in `verify.c` applies the matrix by *columns*:

$$\begin{aligned} R' &= [a] P + [c] Q & (\text{column 0: } \text{mat}[0][0], \text{mat}[1][0]) \\ S' &= [b] P + [d] Q & (\text{column 1: } \text{mat}[0][1], \text{mat}[1][1]) \end{aligned}$$

where $\mathbf{M} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. An implementor who applies by rows (the natural reading of “ $\mathbf{M} \cdot (P, Q)^T$ ”) will produce wrong results.

Impact. Applying by rows instead of columns silently produces the wrong torsion basis, causing the (2,2)-isogeny chain in verification to fail (typically a panic in the splitting step).

Recommendation. State explicitly in Algorithm 4.8 and Algorithm 4.9 that the matrix is applied by columns, or equivalently define the multiplication as $(P', Q') = (P, Q) \cdot \mathbf{M}^T$.

C GENERALIZEDREPRESENTINTEGER sampling ranges and divisibility

Affects Algorithm 3.12 (GENERALIZEDREPRESENTINTEGER), and transitively Algorithm 3.15 (FIXEDDEGREEISOGENY) during signing. MEDIUM

Algorithm 3.12 describes sampling z from $[1, \dots, m]$ (line 4) and t from $[-m', \dots, m']$ (line 5). Two aspects of the algorithm are easy to implement incorrectly:

1. Sign of t matters for the isogeny condition. Since $M' = 4M - p(z^2 + qt^2)$ depends on t^2 , it is tempting to sample only $t \geq 0$ and rely on the symmetry. However, the isogeny condition (line 15) checks $x - t \equiv 2 \pmod{4}$, which *does* depend on the sign of t . Restricting to $t \geq 0$ halves the effective search space for the isogeny condition, which can cause the algorithm to exhaust its bound without finding a solution.

2. Divisibility by 2 in O (line 18) is not coordinate-wise. Line 18 says “set d to be the biggest scalar such that $\gamma/d \in O$ ” and line 19 checks $d = 2$. For the standard order $O_0 = \mathbb{Z}[i] + j\mathbb{Z}[i]$ with $q = 1$ and $\omega = i$, an element $\gamma = x + iy + jz + kt$ satisfies $\gamma/2 \in O_0$ if and only if x, y, z, t are all even. However, for orders with half-integer basis elements (e.g., $O_0 = \mathbb{Z} + i\mathbb{Z} + \frac{1+j}{2}\mathbb{Z} + \frac{i+k}{2}\mathbb{Z}$, which is the actual O_0 for SQIsign), divisibility by 2 in O_0 is *not* equivalent to all coordinates being even in the $\{1, i, j, k\}$ basis. The correct check must account for the order’s common denominator and basis structure.

An implementor who reads “ $\gamma/d \in O$ ” as “all coordinates divisible by d ” (which is correct for $\mathbb{Z}^4$ but not for a non-trivial order) will write code that rejects valid solutions.

Recommendation. Add a remark after line 18 of Algorithm 3.12 clarifying:

The divisibility check “ $\gamma/d \in O$ ” is with respect to the order O , not coordinate-wise in the $\{1, i, j, k\}$ basis. For orders with a non-trivial denominator d_O , the integral coordinates of γ in the order’s column basis must all be divisible by $d \cdot d_O$.

Also note explicitly that the range $[-m', m']$ on line 5 includes negative values, and that both signs must be tried to satisfy the isogeny condition on line 15.

3. Product order in γ construction (line 17). Line 17 writes $\gamma \leftarrow (x + \omega y + jz + \omega jt)$. In quaternion algebra, $\omega j \neq j\omega$ in general ($ij = k$ but $ji = -k$). The C reference’s `quat_order_elem_create` computes $j \cdot \omega \cdot t$ (i.e., left-multiplying by j then by ω), which for $\omega = i$ gives $ji \cdot t = -kt$. Reading line 17 as $\omega jt = ij \cdot t = kt$ produces the opposite sign on the k component.

The difference does not affect the norm (`nrd` depends only on $|\text{components}|^2$), but it *does* affect the isogeny condition (line 15) and the divisibility check (line 18), since both depend on the parity and congruence class of the coordinates.

State whether line 17 means $(\omega j)t$ or $(j\omega)t$, or equivalently state the product order used by `quat_order_elem_create`.

D Response lattice: product vs. intersection

Affects Algorithm 4.2 (SQISIGN.SIGN), line 14.

High

Algorithm 4.2 line 14 writes

$$\alpha_{\text{rsp}} \leftarrow \text{RANDEQUIVALENTQUATERNION}(\overline{I_{\text{com}}} \cap I_{\text{sk}} \cdot I_{\text{chl}})$$

The notation “ $I_{\text{sk}} \cdot I_{\text{chl}}$ ” is ambiguous: it could mean the *ideal product* (the $\mathbb{Z}$ -span of all $\alpha\beta$ with $\alpha \in I_{\text{sk}}, \beta \in I_{\text{chl}}$) or the *intersection* $I_{\text{sk}} \cap I_{\text{chl}}$.

The C reference (sign.c:81–85) computes:

1. $I_{\text{chl,secret}} = I'_{\text{chl}} \cap I_{\text{sk}}$ (intersection, **not** product)
2. $\overline{I_{\text{com}}}$ (conjugate of I_{com})
3. $\alpha_{\text{rsp}} \leftarrow \text{SAMPLE}(I_{\text{chl,secret}} \cap \overline{I_{\text{com}}})$

Furthermore, the C ref intersects the *raw* challenge ideal I'_{chl} (before pushforward) with I_{sk} , not the pushed-forward $I_{\text{chl}} = [I_{\text{sk}}]_* I'_{\text{chl}}$.

An implementor who reads “ $I_{\text{sk}} \cdot I_{\text{chl}}$ ” as an ideal product will compute a completely different lattice. The ideal product’s norm is $\text{nrd}(I_{\text{sk}}) \cdot \text{nrd}(I_{\text{chl}}) \approx 2^{381}$, while the intersection’s norm divides $\text{lcm}(\text{nrd}(I_{\text{sk}}), \text{nrd}(I_{\text{chl}}))$. Sampling from the wrong lattice produces elements that generate trivial ideals (the HNF collapses to a scaled copy of $\mathcal{O}_0$), causing the response phase to fail silently.

Recommendation. Replace “ $I_{\text{sk}} \cdot I_{\text{chl}}$ ” with explicit “ $I_{\text{sk}} \cap I'_{\text{chl}}$ ” and note that the intersection uses the pre-pushforward challenge ideal.

E FIXEDDEGREEISOGENY missing u^{-1} normalization

Affects Algorithm 3.15 (FIXEDDEGREEISOGENY), and transitively Algorithm 3.13 (IDEALTOISOGENY) during signing.

CRITICAL

Algorithm 3.15 line 2 computes $\theta \leftarrow \text{REPRESENTINTEGER}(u \cdot (2^e - u), \mathcal{O}_t, \text{true})$, and line 4 constructs the $(2, 2)$ -isogeny kernel from $([u]P, \theta(P))$ and $([u]Q, \theta(Q))$. Since $\text{nrd}(\theta) = u \cdot (2^e - u)$, the endomorphism θ has degree $u(2^e - u)$; the kernel we want is that of the degree- $(2^e - u)$ isogeny, generated by $(P, (\theta/u)(P))$.

The C reference multiplies all four quaternion coordinates of θ by $u^{-1} \bmod 2^{e+2}$ before applying the endomorphism to the torsion basis (dim2id2iso.c, lines 133–143). Without this normalization, the ACTIONBYTRANSLATION determinant of the gluing kernel is degenerate: the splitting step finds zero (or ten) vanishing $U_{i,j}(0)$ indices instead of exactly one, and the $(2, 2)$ -chain fails.

Current spec text. Algorithm 3.15 line 4 writes “ $K_1 = (u \cdot P_t, \theta(P_t))$ ” without mentioning the u^{-1} normalization. The pseudocode is algebraically suggestive (the u -torsion is killed after $f-2-e$ doublings), but in practice the non-normalized kernel is numerically degenerate and no $(2, 2)$ -chain succeeds.

Recommendation. Replace line 4 of Algorithm 3.15 with $(P_t, (\theta \cdot u^{-1} \bmod 2^{e+2})(P_t))$ and note that u is odd (hence invertible $\bmod 2^{e+2}$) and that the extra $+2$ in the modulus accounts for the gluing’s order-4 structure.

F Fixed-precision Hermite Normal Form

Affects Algorithm 3.2 (HNF), and transitively every algorithm that constructs or reduces an ideal lattice — including Algorithm 3.9 (RANDEQUIVALENTPRIMEIDEAL), Algorithm 3.10 (RANDOMIDEALGIVENORM), and Algorithm 3.15 (FIXEDDEGREEISOGENY).

High

Algorithm 3.2 describes the column-style Hermite Normal Form via extended-GCD elimination. The pseudocode operates over $\mathbb{Z}$, i.e., arbitrary-precision integers.

What goes wrong. Classical HNF’s intermediate entries can grow exponentially in the rank. For rank-4 quaternion ideal lattices at NIST-I the initial column entries can reach $\sim 2^{770}$ bits (the $p \cdot g_i$ products from Algorithm 3.10). After a few xgcd elimination steps, intermediate column entries exceed any reasonable fixed-width budget. At our storage width of 1920 bits (30×64), the resulting “HNF” silently wraps: we observed basis columns collapse to elements of the ambient order O_0 (reduced norms of 4 and $\sim 2^{251}$) rather than the intended ideal I . Every subsequent operation — `RANDEQUIVALENTPRIMEIDEAL`, `IDEALTOISOGENY`, the entire response phase — then operates on a lattice unrelated to I .

What the C reference does. The C reference uses GMP (`ibz_t`), so coefficient growth is absorbed by arbitrary-precision arithmetic. The classical algorithm works correctly there.

Fix. Use the standard Domich–Kannan–Trotter modular HNF (Cohen, *A Course in Computational Algebraic Number Theory*, §2.4.2): given a positive multiple D of the lattice determinant $\det(L)$, reduce every intermediate entry modulo D after every arithmetic update. This bounds entries by D instead of by the exponential classical bound, at the cost of computing at a wider intermediate width $W \geq 2 \cdot \lceil \log_2 D \rceil / 64$. The result is identical to the classical HNF in $\mathbb{Z}$.

For NIST-I commitment ideals, $D = 4d^4N^2p \approx 2^{1282}$ is a compile-time constant. For response-phase and post-reduce ideals the modulus varies and is computed per call.

Recommendation. Add a remark after Algorithm 3.2 noting that fixed-precision implementations must account for coefficient growth, and that the standard modular HNF technique (reducing modulo a known multiple of the lattice determinant) is sufficient to keep intermediates bounded.

G Endomorphism image $P-Q$ and the Okeya-Sakurai lift

Affects Algorithm 3.15 (`FIXEDDEGREEISOGENY`), and transitively `IDEALTOISOGENY` and `KEYGEN`. **HIGH**

Algorithm 3.15 line 4 says to construct the kernel from $(\theta(P), \theta(Q))$. The $(2, 2)$ -isogeny chain lifts these Montgomery x -only points to Jacobian (x, y, z) coordinates via the Okeya-Sakurai formula, which requires the projective x -only representative of $P-Q$ (not just its affine x -coordinate).

The Okeya-Sakurai formula recovers y_Q from $(x_P, y_P, X_Q, Z_Q, X_{P-Q}, Z_{P-Q})$. The specific projective representative $(X_{P-Q} : Z_{P-Q})$ determines the recovered y_Q . Two representatives that agree in the ratio X/Z but differ in the individual coordinates produce different y_Q values—one correct, one not.

The spec does not specify how $\theta(P-Q)$ is obtained. Two natural approaches:

1. Compute $\theta(P-Q)$ as a third biladder call with scalars $(m_{00}-m_{01}, m_{10}-m_{11})$ on the same basis triple $(P, Q, P-Q)$.
2. Compute `DIFFERENCEPOINT`($\theta(P), \theta(Q)$), which takes an $\mathbb{F}_{p^2}$ square root and is ambiguous ($P-Q$ vs. $P+Q$).

Approach (1) produces three outputs whose projective representatives are consistent (all derived from the same basis via differential addition), so the Okeya-Sakurai lift recovers the correct y . Approach (2) picks a deterministic but potentially wrong branch of the square root, causing the lift to recover the wrong y for roughly half of all inputs. In our implementation, approach (2) caused the $(2, 2)$ -chain to fail for every θ from `REPRESENTINTEGER` while succeeding for the i automorphism (a degenerate case whose square root happens to land on the correct branch).

Recommendation. Add a remark in Algorithm 3.15 or in the description of the $(2, 2)$ -isogeny chain that the difference $\theta(P) - \theta(Q)$ must be computed by applying θ to $P - Q$ directly (via the biladder with the same basis triple), *not* by recomputing DIFFERENCEPOINT on the outputs. The same constraint applies to every other site that constructs a basis triple for the Okeya-Sakurai lift.